

Email - 'next-word'_prediction

Importing the Libraries

In [1]: `!pip install wordcloud`

```
Requirement already satisfied: wordcloud in c:\users\home\anaconda3\lib\site-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in c:\users\home\anaconda3\lib\site-packages (from wordcloud) (2.3.5)
Requirement already satisfied: pillow in c:\users\home\anaconda3\lib\site-packages (from wordcloud) (12.0.0)
Requirement already satisfied: matplotlib in c:\users\home\anaconda3\lib\site-packages (from wordcloud) (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.3.3)
Requirement already satisfied: cyclor>=0.10 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (25.0)
Requirement already satisfied: pyparsing>=3 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\home\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)
```

```
In [44]: import os
import email
import pandas as pd
import numpy as np
from tqdm import tqdm
import gc
from os import path
import re
import csv

from wordcloud import WordCloud, STOPWORDS
from nltk.tokenize import sent_tokenize
from nltk.translate.bleu_score import sentence_bleu

import nltk
import random
from nltk import bigrams, trigrams
from collections import Counter, defaultdict
from multiprocessing import Pool
from nltk.corpus import reuters
```

In []:

Uploading the dataset and creating a sample

```
In [3]: df = pd.read_csv("emails.csv")
df
```

```
Out[3]:
```

	file	message
0	allen-p/_sent_mail/1.	Message-ID: <18782981.1075855378110.JavaMail.e...
1	allen-p/_sent_mail/10.	Message-ID: <15464986.1075855378456.JavaMail.e...
2	allen-p/_sent_mail/100.	Message-ID: <24216240.1075855687451.JavaMail.e...
3	allen-p/_sent_mail/1000.	Message-ID: <13505866.1075863688222.JavaMail.e...
4	allen-p/_sent_mail/1001.	Message-ID: <30922949.1075863688243.JavaMail.e...
...	...	...
517396	zufferli-j/sent_items/95.	Message-ID: <26807948.1075842029936.JavaMail.e...
517397	zufferli-j/sent_items/96.	Message-ID: <25835861.1075842029959.JavaMail.e...
517398	zufferli-j/sent_items/97.	Message-ID: <28979867.1075842029988.JavaMail.e...
517399	zufferli-j/sent_items/98.	Message-ID: <22052556.1075842030013.JavaMail.e...
517400	zufferli-j/sent_items/99.	Message-ID: <28618979.1075842030037.JavaMail.e...

517401 rows × 2 columns

In []:

```
In [4]: df = df.sample(frac=0.35, random_state=42)
df.reset_index(inplace = True, drop = True)
```

```
In [5]: df.head()
```

Out[5]:

	file	message
0	shackleton-s/sent/1912.	Message-ID: <21013688.1075844564560.JavaMail.e...
1	farmer-d/logistics/1066.	Message-ID: <22688499.1075854130303.JavaMail.e...
2	parks-j/deleted_items/202.	Message-ID: <27817771.1075841359502.JavaMail.e...
3	stokley-c/chris_stokley/iso/client_rep/41.	Message-ID: <10695160.1075858510449.JavaMail.e...
4	germany-c/all_documents/1174.	Message-ID: <27819143.1075853689038.JavaMail.e...

In [6]: `df.shape`

Out[6]: (181090, 2)

We can see that there are total 1,81,090 emails in data after sampling

In []:

Example of this dataset

In [7]: `df['message'][0]`

Out[7]: 'Message-ID: <21013688.1075844564560.JavaMail.evans@thyme>\nDate: Tue, 29 Aug 2000 01:26:00 -0700 (PDT)\nFrom: sara.shackleton@enron.com\nTo: william.bradford@enron.com\nSubject: Re: Credit Derivatives\nMime-Version: 1.0\nContent-Type: text/plain; charset=us-ascii\nContent-Transfer-Encoding: 7bit\nX-From: Sara Shackleton\nX-To: William S Bradford\nX-cc: \nX-bcc: \nX-Folder: \\Sara_Shackleton_Dec2000_June2001_1\\Notes Folders\\Sent\nX-Origin: SHACKLETON-S\nX-FileName: sshackle.nsf\n\nBill: Thanks for the info. I also spoke with Jeff about how \nEnronCredit.com Ltd. was going to work since Dennis O'Connell (London lawyer) \nis responsible for that group. Maybe you will be able to clarify which of \nJeff\'s "positions" will be hedged and which will be backed to EnronCredit. \nMaybe Rod will be handling most of Jeff\'s credit. I\'d appreciate an update. \nSara\n\n\nWilliam S Bradford\n08/29/2000 07:24 AM\nTo: Sara Shackleton/HOU/ECT@ECT\ncc: Mark Taylor/HOU/ECT@ECT, Paul Rados/Corp/Enron@ENRON, Rod Nelson/LON/ECT@ECT\nSubject: Re: Credit Derivatives\n\nSara,\n\nPlease contact either Paul Rados or me on credit derivatives in the U.S. \nRod Nelson is the lead credit support for EnronCredit.com and should also be \navailable in London, if necessary. I am not aware of these recent trades but \nI am having lunch with Jeff Kinneman on Thursday to discuss among other \nthings Credit support for his business. \n\nIt does concern me that we would offer to provide collateral DLJ without \nTreasury\'s approval.\n\nBill\n\n\n\n\n\n\n\nFrom: Sara Shackleton on 08/28/2000 06:53 PM\nTo: William S Bradford/HOU/ECT@ECT\ncc: Mark Taylor/HOU/ECT@ECT\nSubject: Credit Derivatives\n\nI am seeing more and more credit derivatives. The trades originating in \nHouston are coming from Jeff Kinneman\'s bond/debt traders but are booked in \nthe name of ECT Investments, Inc. or ENA. I can\'t be certain if these are \nbacked to EnronCredit.com Limited. \n\nIn particular, there is a proposed total return bond trade with DLJ \nInternational Capital requiring ECT Investments to post collateral. We have \nno master with this party. Who in Houston credit is looking at credit \nderivatives?\n\nFYI, other credit deals seem to be transacted with Deutsche Bank and UBS AG.\n\n\n\n'

Extracting the content of these messages

```
In [8]: def body(messages):
        column = []
        for message in messages:
            e = email.message_from_string(message)
            column.append(e.get_payload())
        return column

df['content'] = body(df['message'])
```

```
In [9]: df.head()
```

Out[9]:

	file	message	content
0	shackleton-s/sent/1912.	Message-ID: <21013688.1075844564560.JavaMail.e...	Bill: Thanks for the info. I also spoke wit...
1	farmer-d/logistics/1066.	Message-ID: <22688499.1075854130303.JavaMail.e...	Aimee,\nPlease check meter #1591 Lamay gas lif...
2	parks-j/deleted_items/202.	Message-ID: <27817771.1075841359502.JavaMail.e...	GCCA Crawfish and rip-off raffle & over-priced...
3	stokley-c/chris_stokley/iso/client_rep/41.	Message-ID: <10695160.1075858510449.JavaMail.e...	<<Keoni.zip>> Chris, per your request here ar...
4	germany-c/all_documents/1174.	Message-ID: <27819143.1075853689038.JavaMail.e...	I'm trying to change the Receipt Meter on deal...

In []:

Data PreProcessing

Remove all non-alphabetic characters

Convert all content to lower case

And remove extra spaces at the beginning and the end

```
In [10]: df.content = df.content.str.replace("\n", "") ## NewLine Characters
df.content = df.content.apply(lambda x: re.sub('[^A-Za-z0-9]+', ' ', x)) ## Remove
df.content = df.content.str.lower() ## Convert to Lowercase
df.content = df.content.str.strip() ## Remove Extra Spaces at Ends
```

```
In [11]: df.head()
```

Out[11]:

	file	message	content
0	shackleton-s/sent/1912.	Message-ID: <21013688.1075844564560.JavaMail.e...	bill thanks for the info i also spoke with jef...
1	farmer-d/logistics/1066.	Message-ID: <22688499.1075854130303.JavaMail.e...	aimee please check meter 1591 lamay gas lift i...
2	parks-j/deleted_items/202.	Message-ID: <27817771.1075841359502.JavaMail.e...	gccca crawfish and rip off raffle over priced p...
3	stokley-c/chris_stokley/iso/client_rep/41.	Message-ID: <10695160.1075858510449.JavaMail.e...	keoni zip chris per your request here are the ...
4	germany-c/all_documents/1174.	Message-ID: <27819143.1075853689038.JavaMail.e...	i m trying to change the receipt meter on deal...

In []:

Tokenization - breaking the content into an array of tokens

```
In [12]: df['body_list'] = df.content.apply(lambda x: x.split())
```

```
In [13]: df.head()
```

Out[13]:

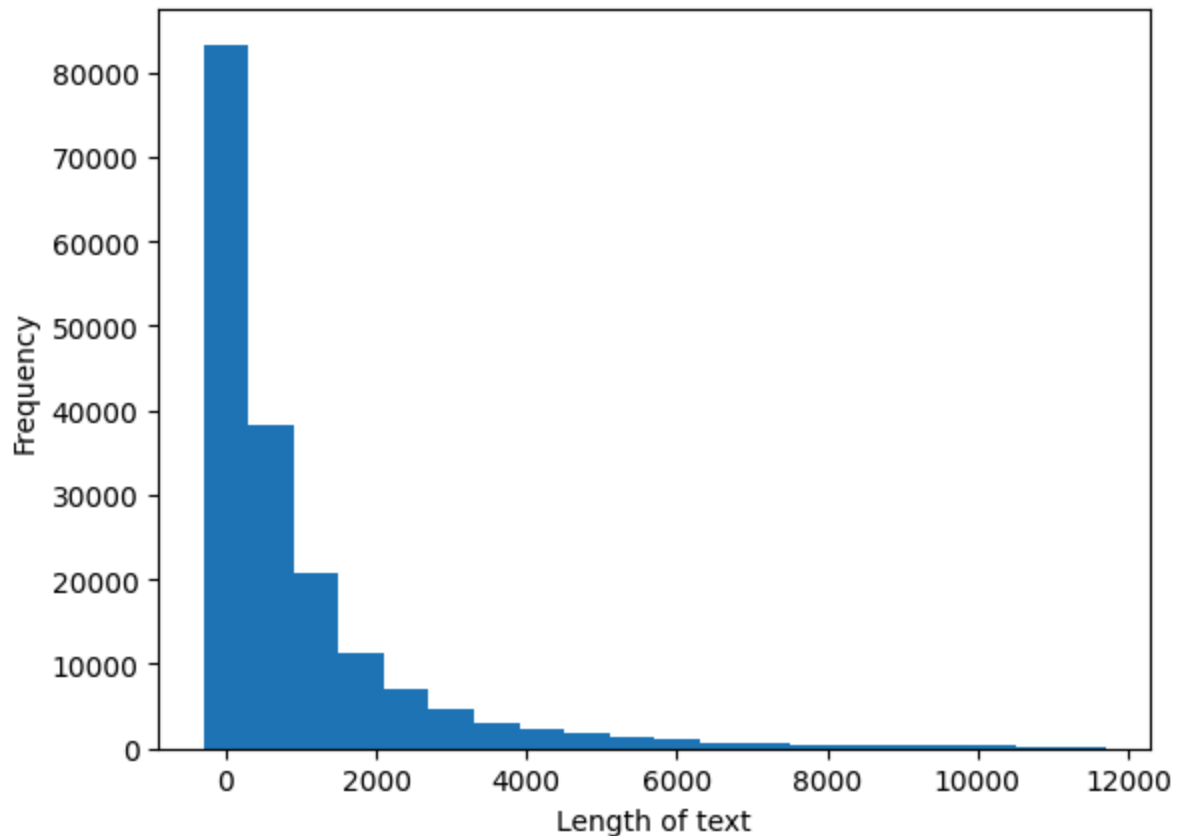
	file	message	content	body_list
0	shackleton-s/sent/1912.	Message-ID: <21013688.1075844564560.JavaMail.e...	bill thanks for the info i also spoke with jef...	[bill, thanks, for, the, info, i, also, spoke,...
1	farmer-d/logistics/1066.	Message-ID: <22688499.1075854130303.JavaMail.e...	aimee please check meter 1591 lamay gas lift i...	[aimee, please, check, meter, 1591, lamay, gas...
2	parks-j/deleted_items/202.	Message-ID: <27817771.1075841359502.JavaMail.e...	gcc crawfish and rip off raffle over priced p...	[gcc, crawfish, and, rip, off, raffle, over, ...
3	stokley-c/chris_stokley/iso/client_rep/41.	Message-ID: <10695160.1075858510449.JavaMail.e...	keoni zip chris per your request here are the ...	[keoni, zip, chris, per, your, request, here, ...
4	germany-c/all_documents/1174.	Message-ID: <27819143.1075853689038.JavaMail.e...	i m trying to change the receipt meter on deal...	[i, m, trying, to, change, the, receipt, meter...

In []:

EDA

```
In [14]: if __name__ == "__main__":  
# EDA for text length  
x = list(df['content'].apply(lambda x:len(x)))
```

```
plt.hist(x,range=[1,12000],bins = 20,align = 'left')
plt.rcParams["figure.figsize"] = (10,4)
plt.xlabel("Length of text")
plt.ylabel("Frequency")
```



****Observations**** - From above plot it is clear that length of text ranges from 0 to 1594315 - Distribution for the length of text is looks like power-law distribution Which has frequency for sort sentences higher than long sentences - As we can see above from percentile values 90 percent of text has max length 3277 chars

In []:

Visualize Email content using word cloud

```
In [15]: if __name__ == "__main__":
# Word cloud for content , X-TO, X-From
wordcloud = WordCloud(width = 800, height = 800,
                      background_color = 'white',
                      stopwords = STOPWORDS ,
                      max_font_size = 50,
                      min_font_size = 10).generate(df['content'].to_string())
# plot the WordCloud image
plt.figure(figsize = (8, 8), facecolor = None)
plt.imshow(wordcloud)
plt.axis("off")
plt.tight_layout(pad = 0)

plt.show()
```



In [16]:

9/26

```
        counts[word] = 1  
    return counts
```

```
In [17]: if __name__ == "__main__":  
    # most 100 frequent words in content  
    most_frequent_words = Counter(word_count(str(df['content'].to_string()))).most_co  
    print('\n'.join(map(str, most_frequent_words)))  
    most_frequent_words = None  
    del(most_frequent_words) ## Deleting the variable - 'most_frequent_words'  
    gc.collect() ##free memory
```

('the', 55709)
('...', 32002)
('you', 27140)
('for', 18368)
('forwarded', 17007)
('this', 16368)
('and', 13695)
('from', 12579)
('have', 12192)
('please', 12119)
('attached', 10966)
('are', 10647)
('that', 9848)
('ect', 9708)
('hou', 9685)
('enron', 8996)
('message', 8887)
('your', 8548)
('original', 7973)
('with', 7878)
('will', 7203)
('thanks', 6452)
('here', 5932)
('fyi', 4770)
('just', 4749)
('has', 4705)
('can', 4506)
('sent', 4350)
('not', 4275)
('all', 4189)
('our', 4036)
('see', 3987)
('know', 3934)
('was', 3934)
('image', 3850)
('date', 3783)
('t...', 3664)
('2...', 3574)
('vince', 3536)
('what', 3455)
('start', 3402)
('mark', 3399)
('hourahead', 3393)
('would', 3364)
('jeff', 3309)
('hour', 3275)
('been', 3158)
('get', 3049)
('good', 2986)
('dear', 2952)
('com', 2921)
('about', 2905)
('out', 2852)
('call', 2842)
('corp', 2830)
('like', 2759)

```
( 'today', 2690)
( 'following', 2660)
( 'new', 2572)
( 'a...', 2521)
( 'let', 2520)
( 'meeting', 2509)
( 'need', 2497)
( 'find', 2483)
( 'there', 2447)
( '2000', 2408)
( 'think', 2348)
( 'don', 2282)
( '2001', 2282)
( 'but', 2278)
( 'john', 2211)
( 's...', 2200)
( 'email', 2113)
( 'received', 2035)
( 'http', 2028)
( 'any', 2024)
( 'kaminski', 2013)
( 'one', 1955)
( 'how', 1950)
( 'time', 1938)
( 'th...', 1935)
( 'some', 1923)
( 'thank', 1894)
( 'kay', 1892)
( 'i...', 1863)
( 'mail', 1852)
( 'deal', 1843)
( 'below', 1784)
( 'going', 1777)
( 'sorry', 1775)
( 'report', 1765)
( 'steve', 1759)
( 'had', 1734)
( 'draft', 1730)
( 'mike', 1689)
( 'did', 1680)
( 'note', 1677)
( 'sara', 1665)
( 'request', 1659)
( 'per', 1658)
```

In []:

Train - Test split

```
In [18]: from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(df, test_size=0.3, random_state=42)

## resetting the train_set/test_set_index
```

```
train_set.reset_index(inplace=True, drop=True)
test_set.reset_index(inplace=True, drop=True)
```

In []:

Next Word prediction

```
In [19]: df = train_set.copy()
df.head()
```

Out[19]:

	file	message	content	
0	kean-s/attachments/1259.	Message-ID: <21917855.1075851032223.JavaMail.e...	fyi jade eaton is one of lara s contacts at do...	[
1	ward-k/inbox/401.	Message-ID: <27507725.1075862959396.JavaMail.e...	kim attached are the frito lay special provisi...	[
2	kaminski-v/discussion_threads/4043.	Message-ID: <19700597.1075856428881.JavaMail.e...	forwarded by vince j kaminski hou ect on 04 27...	
3	watson-k/sent_items/53.	Message-ID: <5069760.1075852891455.JavaMail.ev...	jean let s book lunch on the 10th let s talk e...	[j
4	white-s/meetings/691.	Message-ID: <7196473.1075858789589.JavaMail.ev...	calendar entry appointmentdescription sara bol...	ap

```
In [20]: corpus = df['body_list'].to_list()
```

In []:

Creating a bi-gram model

```
In [21]: x = defaultdict(lambda: defaultdict(lambda: 0))
```

```
In [22]: for w1, w2 in bigrams(corpus[0], pad_right=True, pad_left=True):
print(w1,w2)
x[(w1)][w2] += 1
```

None fyi
fyi jade
jade eaton
eaton is
is one
one of
of lara
lara s
s contacts
contacts at
at doj
doj forwarded
forwarded by
by steven
steven j
j kean
kean na
na enron
enron on
on 03
03 02
02 2001
2001 01
01 12
12 pm
pm lara
lara leibman
leibman enron
enron communications
communications 02
02 28
28 2001
2001 03
03 23
23 pm
pm to
to steven
steven j
j kean
kean na
na enron
enron enron
enron cc
cc richard
richard shapiro
shapiro na
na enron
enron enron
enron subject
subject re
re hpl
hpl closingsteve
closingsteve thanks
thanks for
for forwarding
forwarding actually

actually brian
brian sent
sent it
it to
to me
me on
on monday
monday and
and i
i followed
followed up
up with
with ann
ann and
and barbara
barbara i
i have
have not
not spoken
spoken with
with brian
brian yet
yet because
because i
i am
am waiting
waiting for
for a
a green
green red
red light
light from
from barbara
barbara as
as far
far as
as jade
jade eaton
eaton is
is concerned
concerned barbara
barbara and
and i
i spoke
spoke at
at length
length yesterday
yesterday and
and she
she is
is concerned
concerned about
about the
the counterparty
counterparty s
s reaction

reaction to
to a
a meeting
meeting with
with doj
doj even
even an
an informal
informal one
one she
she plans
plans to
to get
get back
back to
to me
me though
though with
with regard
regard to
to whether
whether they
they want
want me
me to
to get
get involved
involved at
at this
this stage
stage i
i spoke
spoke to
to rick
rick as
as well
well so
so he
he is
is up
up to
to date
date with
with the
the current
current status
status i
i ll
ll let
let you
you know
know if
if anything
anything further
further develops
develops thanks

thanks again
again lara
lara steven
steven j
j kean
kean enron
enron 02
02 28
28 01
01 02
02 13
13 pm
pm to
to lara
lara leibman
leibman enron
enron communications
communications enron
enron communications
communications cc
cc subject
subject hpl
hpl closing
closing forwarded
forwarded by
by steven
steven j
j kean
kean na
na enron
enron on
on 02
02 28
28 2001
2001 02
02 12
12 pm
pm brian
brian redmond
redmond ect
ect 02
02 26
26 2001
2001 02
02 57
57 pm
pm to
to john
john j
j lavorato
lavorato enron
enron enronxgate
enronxgate louise
louise kitchen
kitchen hou
hou ect

ect ect
ect david
david w
w delaine
delaine hou
hou ees
ees ees
ees steven
steven j
j kean
kean na
na enron
enron enron
enron sally
sally beck
beck hou
hou ect
ect ect
ect mark
mark e
e haedicke
haedicke hou
hou ect
ect ect
ect david
david oxley
oxley enron
enron enronxgate
enronxgate wes
wes colwell
colwell hou
hou ect
ect ect
ect tim
tim despain
despain hou
hou ect
ect ect
ect joseph
joseph deffner
deffner enron
enron enronxgate
enronxgate anne
anne c
c koehler
koehler hou
hou ect
ect ect
ect barbara
barbara n
n gray
gray hou
hou ect
ect ect
ect jim
jim coffey

coffey hou
hou ect
ect ect
ect ann
ann ballard
ballard enron
enron enronxgate
enronxgate jim
jim schwieger
schwieger hou
hou ect
ect ect
ect thomas
thomas a
a martin
martin hou
hou ect
ect ect
ect richard
richard shapiro
shapiro na
na enron
enron enron
enron joe
joe allen
allen na
na enron
enron enron
enron tommy
tommy yanowski
yanowski enron
enron enronxgate
enronxgate philippe
philippe a
a bibi
bibi enron
enron enronxgate
enronxgate patrick
patrick wade
wade hou
hou ect
ect ect
ect tanya
tanya rohauer
rohauer enron
enron enronxgate
enronxgate fran
fran l
l mayes
mayes enron
enron enronxgate
enronxgate michelle
michelle cash
cash hou
hou ect
ect ect

ect carolyn
carolyn gilley
gilley hou
hou ect
ect ect
ect robert
robert l
l hall
hall et
et s
s enron
enron enron
enron ranabir
ranabir dutt
dutt enron
enron enronxgate
enronxgate tom
tom shelton
shelton hou
hou ect
ect ect
ect steve
steve hpl
hpl schneider
schneider hou
hou ect
ect ect
ect kenneth
kenneth w
w kaase
kaase hou
hou ect
ect ect
ect cc
cc subject
subject hpl
hpl closingplease
closingplease find
find the
the attached
attached status
status update
update for
for the
the hpl
hpl transaction
transaction if
if you
you have
have any
any questions
questions please
please contact
contact me
me thanks

thanks brian
brian None

```
In [23]: def build_bigram_model(corpus):
# Create a placeholder for model
bigram_model1 = defaultdict(lambda: defaultdict(lambda: 0))

# Count frequency of co-occurrence
for sentence in corpus: # For each email
    for w1, w2 in bigrams(sentence, pad_right=True, pad_left=True):
        bigram_model1[(w1)][w2] += 1

# Let's transform the counts to probabilities
for w1 in bigram_model1:
    #print(w1)
    total_count = float(sum(bigram_model1[w1].values()))
    for w2 in bigram_model1[w1]:
        bigram_model1[w1][w2] /= total_count

return bigram_model1
```

```
In [24]: bigram = build_bigram_model(corpus)
```

```
In [25]: def get_next_word(model, input_string):
    if len(input_string.split()) > 1:
        input_string_ = input_string.split()
        output = dict(model[tuple(input_string_)])
        return "{} {}".format(input_string,
                                list({k: v for k, v in sorted(output.items(), key=lambda item: item[1],
                                                                reverse=True)}))
    else:
        input_string = (input_string)
        output = dict(model[input_string])
        return "{} {}".format(input_string,
                                list({k: v for k, v in sorted(output.items(), key=lambda item: item[1],
                                                                reverse=True)}))
```

```
In [26]: get_next_word(bigram, 'how')
```

Out[26]: 'how to'

```
In [27]: get_next_word(bigram, 'they')
```

Out[27]: 'they are'

```
In [28]: get_next_word(bigram, 'what')
```

Out[28]: 'what is'

```
In [29]: get_next_word(bigram, 'href')
```

Out[29]: 'href http'

In []:

Creating a Tri-gram Model

```
In [30]: def build_trigram_model(corpus):
# Create a placeholder for model
trigram_model = defaultdict(lambda: defaultdict(lambda: 0))

# Count frequency of co-occurrence
for sentence in corpus:
    for w1, w2, w3 in trigrams(sentence, pad_right=True, pad_left=True):
        trigram_model[(w1, w2)][w3] += 1
        # break

# Let's transform the counts to probabilities
for w1_w2 in trigram_model:
    total_count = float(sum(trigram_model[(w1,w2)].values()))
    for w3 in trigram_model[(w1,w2)]:
        trigram_model[(w1,w2)][w3] /= total_count

return trigram_model
```

```
In [31]: trigram_model = build_trigram_model(corpus)
```

```
In [32]: get_next_word(trigram_model, 'i may')
```

```
Out[32]: 'i may be'
```

```
In [33]: get_next_word(trigram_model, 'can we')
```

```
Out[33]: 'can we get'
```

```
In [34]: get_next_word(trigram_model, 'i will')
```

```
Out[34]: 'i will be'
```

```
In [ ]:
```

Perplexity

metric used to evaluate how well language model predicts text.

```
In [35]: input_sentence = "can we get the best of luck in your group that"
tokenized_sentence = input_sentence.split()
pp_w = []
N = len(tokenized_sentence)

tokenized_sentence.insert(0, None)
tokenized_sentence.append(None)
tokenized_sentence
```

```
Out[35]: [None,
          'can',
          'we',
          'get',
          'the',
          'best',
          'of',
          'luck',
          'in',
          'your',
          'group',
          'that',
          None]
```

```
In [36]: pp_w
```

```
Out[36]: []
```

```
In [37]: def perplexity(n_gram, input_sentence):
          tokenized_sentence = input_sentence.split()
          pp_w = []
          N = len(tokenized_sentence)

          if n_gram == 2:
              tokenized_sentence.insert(0, None)
              tokenized_sentence.append(None)
              for i in tokenized_sentence[:-1]:
                  try:
                      pp_w.append(1 / (0.0001+bigram[(i)][tokenized_sentence[
                          tokenized_sentence.index(i)+1]]))
                  except:
                      continue

          if n_gram == 3:
              tokenized_sentence.insert(0, None)
              tokenized_sentence.insert(0, None)
              tokenized_sentence.append(None)
              tokenized_sentence.append(None)
              for i in tokenized_sentence[:-1]:
                  #smoothing applied (added 0.0001)
                  pp_w.append(1/(0.0001+trigram_model[tuple([i, tokenized_sentence[tokeni

          if np.prod(pp_w) != 0:
              return round(np.prod(pp_w)**(1/N), 2)
          else:
              print("Cant find perplexity for the sentence")
```

```
In [38]: perplexity_dict = {'content':[], 'bi-gram_score':[], 'tri-gram_score':[]}
          for i in range(20):
              try:
                  body = test_set['content'][i]
                  tri_gram = perplexity(3, test_set['content'][i])
                  bi_gram = perplexity(2, test_set['content'][i])
                  perplexity_dict['content'].append(body)
```

```

perplexity_dict['bi-gram_score'].append(bi_gram)
perplexity_dict['tri-gram_score'].append(tri_gram)
except Exception as e:
    perplexity_dict['content'].append(body)
    perplexity_dict['bi-gram_score'].append(e)
    perplexity_dict['tri-gram_score'].append(e)
df2 = pd.DataFrame(perplexity_dict)
df2

```

Cant find perplexity for the sentence

C:\Users\Home\anaconda3\Lib\site-packages\numpy_core\fromnumeric.py:86: RuntimeWarning: overflow encountered in reduce
 return ufunc.reduce(obj, axis, dtype, out, **passkwargs)

Cant find perplexity for the sentence

Cant find perplexity for the sentence

Out[38]:

	content	bi-gram_score	tri-gram_score
0	forwarded by jane m tholt hou ect on 11 27 200...	27.52	0.04
1	kelly raised this concern too the system is se...	inf	NaN
2	some thoughts 1 we argue against the pjw insta...	inf	NaN
3	mr christman thank you for your invitation for...	109.35	0.05
4	we are pleased to announce organizational chan...	inf	0.03
5	miriam sounds like a great growth opportunity ...	159.44	3.88
6	simon did a nice job on the presentation but i...	260.47	1.04
7	550480 has been changed to merrill per sean cr...	115.74	0.48
8	version 2 0 of the clickathome portal is now a...	101.32	0.02
9	attached is ferc s press release on southern r...	inf	2.75
10	sally where do i go inside enron to get straig...	inf	0.22
11	no changes the first major arctic air mass of ...	inf	inf
12	is there supposed to be something attached geo...	121.26	1.17
13	forwarded by kaye ellis hou ect on 11 03 2000 ...	inf	NaN
14	here s a simple first draft i didn t know how ...	78.13	0.02
15	see attached file sweets wedding letter doc se...	44.81	0.01
16	fyi original message from mack iris sent tuesd...	132.44	2.88
17	1400 smith room 3882ahouston tx 77002 kim knox...	57.14	0.05
18	ken lay and jeff skilling were interviewed on ...	119.83	0.02
19	here is the latest version of the comments we ...	106.73	0.45

Interpretation: - Low Perplexity means good prediction and High Perplexity means Poor prediction. - 'inf' reflect Model is completely failing due to unknown word combinations(n-grams) making the probability becomes 0. - log(0) occurs

mathematically. - The trigram model achieved significantly lower perplexity scores compared to the bigram model, indicating better predictive capability due to the use of additional contextual information.

Creating an n-gram model

```
In [39]: del bigram
del trigram_model
del df
del train_set
del STOPWORDS
gc.collect()
```

Out[39]: 0

```
In [40]: def build_ngram_model(corpus, ngrams=2):
ngram_model = defaultdict(lambda: defaultdict(lambda: 0))

for sentence in tqdm(corpus):
    for i_ngram in range(ngrams, len(sentence)):
        ngram_model[tuple(sentence[i_ngram - ngrams + 1 : i_ngram])][sentence[i_ngram]]

for words in ngram_model:
    total_count = float(sum(ngram_model[words].values()))
    for w2 in ngram_model[words]:
        ngram_model[words][w2] /= total_count

return ngram_model

ngram = build_ngram_model(corpus, ngrams=4)
```

100% | 1267
63/126763 [13:46<00:00, 153.37it/s]

```
In [49]: def complete_sentence(model, text, n_words=10, ngrams=4):

current_text = text

for i in range(n_words):

    # Split sentence into words
    words = current_text.split()

    # Take Last (ngrams - 1) words
    context = tuple(words[-(ngrams-1):])

    # Stop if context not found
    if context not in model:
        print("Context not found:", context)
        break

    # Get next possible words
    possible_words = model[context]

    # Select highest probability word
    next_word = max(possible_words, key=possible_words.get)
```

```
# Append next word
current_text += " " + next_word

# Print each step
print(f"Step {i} {current_text}")

return current_text
```

```
In [50]: complete_sentence(ngram, 'how have you', 10, 4)
```

```
Step 0 how have you been
Step 1 how have you been able
Step 2 how have you been able to
Step 3 how have you been able to get
Step 4 how have you been able to get the
Step 5 how have you been able to get the best
Step 6 how have you been able to get the best price
Step 7 how have you been able to get the best price for
Step 8 how have you been able to get the best price for the
Step 9 how have you been able to get the best price for the transaction
```

```
Out[50]: 'how have you been able to get the best price for the transaction'
```

```
In [ ]:
```

Conclusion: We have modelled the problem of helping people write emails faster as "next-word" prediction problem. Explored Probabilistic Language Models. Auto-completed the sentence by building bi-gram and tri-gram language models. We explored Perplexity as a technique to ascertain the LM's understanding. We saw how Smoothing and interpolation techniques can help overcome the dis-advantages of OOV.

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```